Murang'a University of Technology

Innovation for Prosperity

Unit Code: SCS 414 **Unit Title:** MACHINE LEARNING

Department: COMPUTER SCIENCE

Lecturer's Name: Mr. Munyao C.M.

Lecturer's Tel. No.: 0796313268 **Email Address:** munyao@mut.ac.ke

Credit Hours: 3 Hours

Session: Year 4 Semester 2 **Academic Year:** 2025/2026

Lecture 2

Supervised Learning

What is Supervised Learning?

- **Supervised Learning** is a type of machine learning where the model is trained using **labeled data**, meaning each input has a correct output (label).
- The goal is to learn a **mapping function** from input features to output labels so that the model can predict the correct output for **new, unseen data**.
- Supervised learning is commonly used for **classification** and **regression** problems.
- It is called “supervised” because the model is guided by a **teacher (labels)** during training.

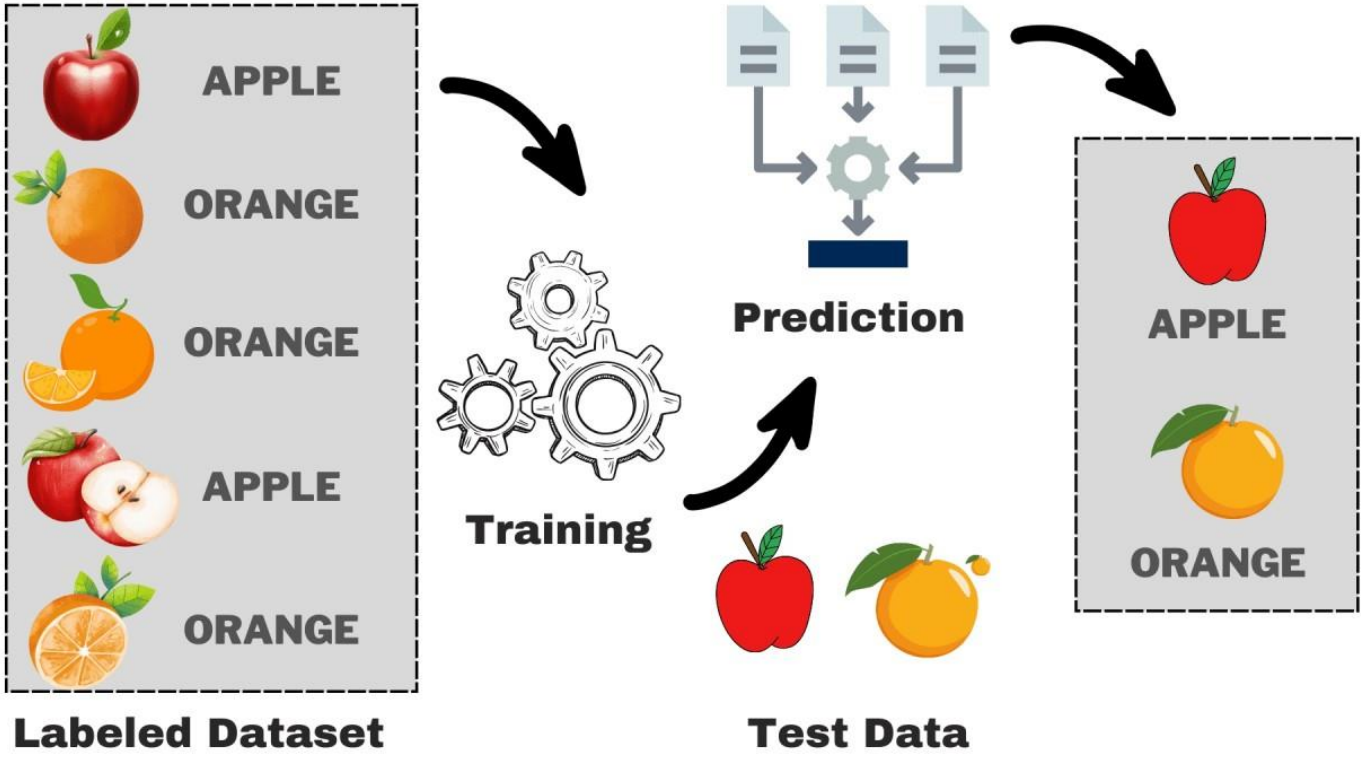
Key Concepts in Supervised Learning

- The following are key concepts you need to understand in supervised learning.
  - **Training Data:** Labeled examples used to teach the model how to make accurate predictions.
  - **Features:** Input variables used for prediction.
  - **Labels:** Output variables or target values the model aims to predict (e.g., price, category, risk level).
  - **Model:** A function or algorithm that maps **features** to **labels** by learning from the training data.
  - **Prediction:** The model's output when given new, unseen data after training.

How Supervised Learning Works

- Supervised learning works by training a model using **labeled data**, where each input is paired with the correct output.
  - **Labeled Training Data:** The process begins with a dataset where every input (e.g., an image of a dog) is paired with its correct label (e.g., “dog”).
  - **Model Training:** An algorithm learns from these input–output pairs, adjusting its internal parameters to reduce errors between its predictions and the actual labels.
  - **Prediction:** Once trained, the model can take new, unlabeled data and predict the correct output, having learned the underlying patterns from the examples.

How Supervised Learning Works



Tutorial: https://youtu.be/g9oESGzcA84

Supervised Learning Tasks

- Supervised learning is used to solve **real-world problems** by learning from labeled data and predicting outcomes.
- The type of **task** depends on the nature of the output required, and this determines the choice of model and evaluation metrics.
- **Regression tasks** involve predicting **continuous numeric values**, where the output is a measurable quantity, whereas, **Classification tasks** involve predicting **discrete categories**, such as whether an email is spam or not, or whether a patient has a disease or not.
- Understanding the task is important because it guides how the **problem is framed**, the **kind of data needed**, and how the model's **performance is assessed**.

Classification Tasks

- **Classification** is a supervised learning task where the goal is to predict **which category or class** an input belongs to.
- The model learns from labeled examples and uses patterns in the data to assign **new inputs** to the correct class.
- It deals with discrete outputs, such as *"yes"* or *"no,"* *"cat"* or *"dog,"* or multiple classes like *"setosa,"* *"versicolor,"* and *"virginica"* in the Iris dataset.
- Classification problems are common in everyday applications such as spam detection and image recognition.
- In later slides, we will explore common classification algorithms, performance metrics, and real-world examples.

Types of Classification Tasks

- Classification problems can be divided into three main types based on the number of classes and labels:
  - **Binary Classification:** The model predicts one of **two classes** (e.g., spam or not spam).
  - **Multiclass Classification:** The model predicts **one class from many classes** (e.g., classifying animals as cat, dog, or bird).
  - **Multilabel Classification:** The model predicts **multiple labels for a single input**, meaning an item can belong to more than one class at the same time (e.g., tagging a photo with “beach”, “sunset”, and “vacation”).
- Each type requires different handling and evaluation because the structure of the output changes.

Classification Algorithms

- **Classification algorithms** are methods used to assign **new data points** into **pre-defined categories or classes** based on patterns learned from labeled training data.
- Different algorithms use different strategies to learn these patterns, such as measuring distance (KNN), building decision rules (Decision Trees), combining multiple models (Random Forest), maximizing class separation (SVM), or using probability theory (Naive Bayes).
- Choosing the right algorithm depends on the **type of data**, **accuracy needs**, and **problem complexity**.
- In the following slides, we will explore each of the five key classification algorithms, how they work, and when to use them.

K-Nearest Neighbors (KNN)

- **K-Nearest Neighbors (KNN)** is a simple and intuitive classification algorithm that assigns a class to a new data point based on the classes of its closest neighbors.
- It operates on the principle that data points with **similar features** are likely to be **close to one another** in a multi-dimensional space (“*birds of a feather flock together”*).
- KNN is **non-parametric** because it does not assume any fixed form or parameters for the data distribution; instead, it makes predictions based directly on the training data.
- KNN works well for small datasets and is easy to understand, but it can become slow and less accurate on large datasets.

How KNN Works

i. **Store training data:** KNN keeps all the labeled examples in memory instead of learning a model.

ii. **Receive a new input:** When a new data point is given, KNN measures the distance between this point and all points in the training data.

iii. **Find nearest neighbors:** It selects the **K closest training points** (neighbors) based on the shortest distance. (Research on at least 3 distance metrics in KNN)

iv. **Majority voting:** The model predicts the class that appears most frequently among the K neighbors.

v. **Output prediction:** The new input is assigned to the class with the highest vote.

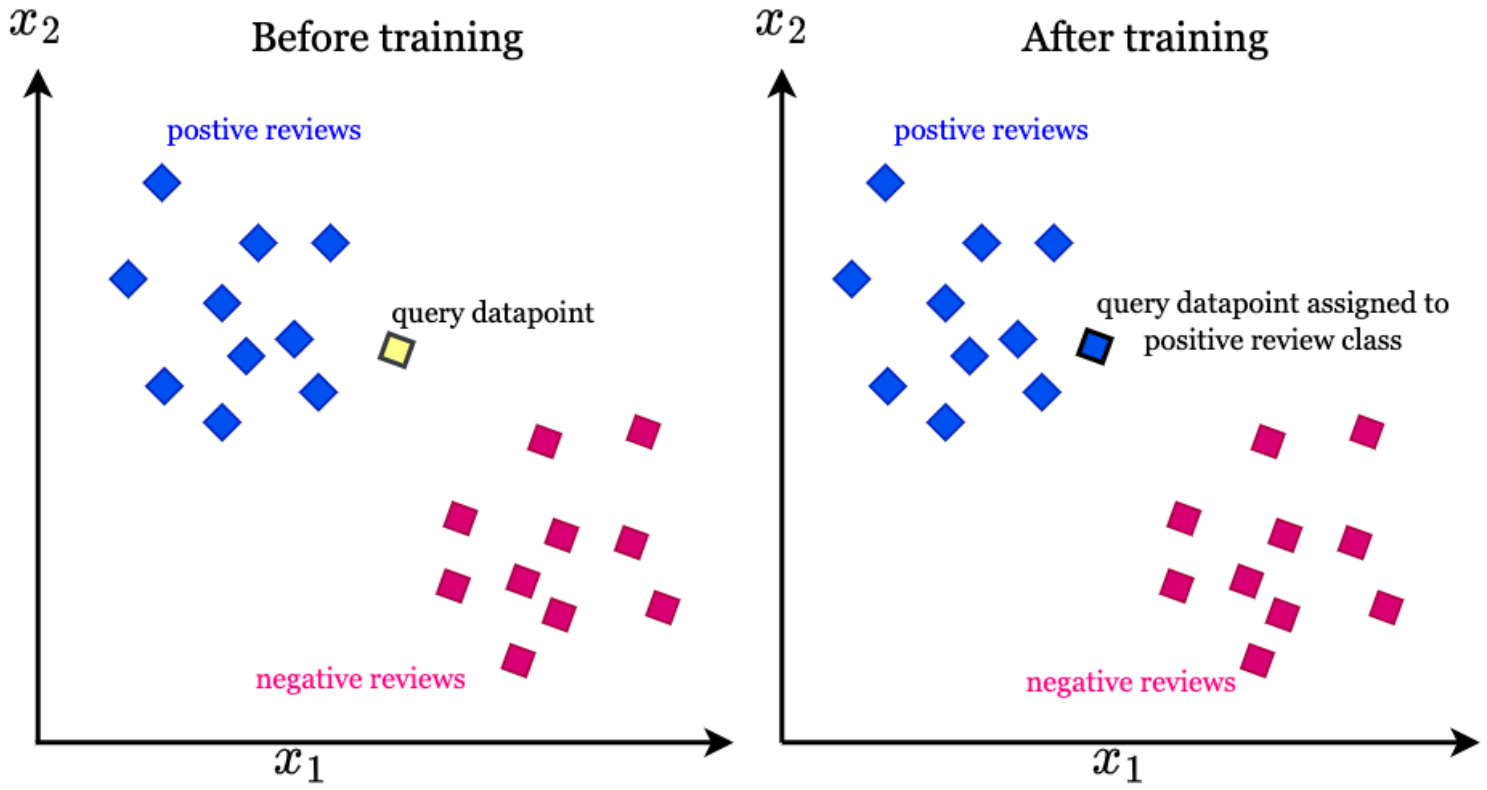
Why KNN is a Lazy Learner

- KNN is called a "**lazy learner"** because it does not actively build a model during training like other algorithms.
  - **No model is built during training:**
    During training, KNN simply stores the labeled data without extracting rules or patterns. There is no learning phase where parameters are adjusted.
  - **Learning happens only during prediction:**
    When a prediction is requested, KNN computes distances to all stored points and finds the nearest neighbors to decide the class. This makes prediction time slow and computationally heavy, especially for large datasets.

Selecting the Value of K in KNN

- The value of **K** is important because it determines how many neighbors influence the final prediction and directly affects the model's accuracy. The following are ways to Select K.
  - **Rule of Thumb:**
    Use K= $\sqrt{n}$ where **n** is the number of training samples. This gives a rough estimate and helps avoid extreme values.
  - **Cross-Validation:**
    Test different K values using validation data and choose the one with the best performance (accuracy or error rate).
  - **Odd Values for K:**
    Choose odd values like 3, 5, or 7 to avoid ties during majority voting.

K-Nearest Neighbors (KNN)



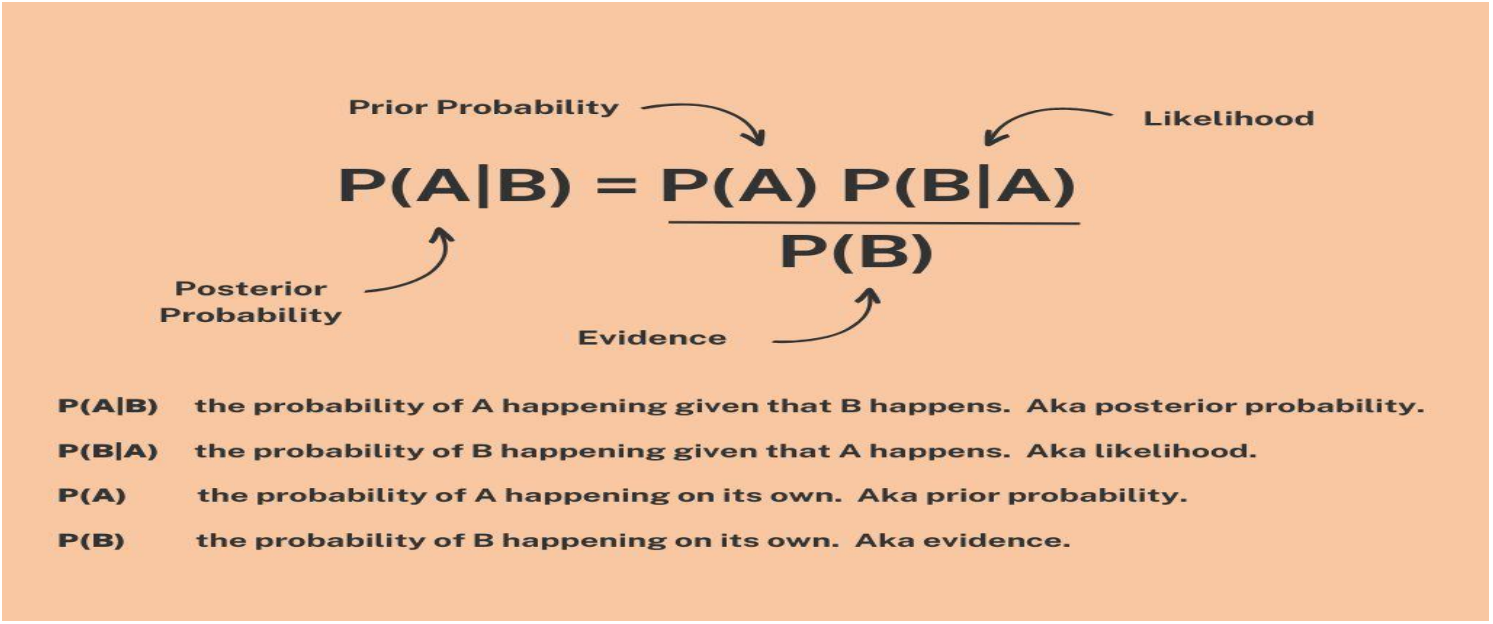
Naive Bayes

- **Naive Bayes** is a **probabilistic classification algorithm** based on **Bayes' Theorem**, developed by **Thomas Bayes**.
- It predicts the class of an input by calculating the **probability of each class given the observed features** and selecting the class with the highest probability.
- Naive Bayes calculates the probability of each class using **Bayes' Theorem** based on the input features.
- Naive Bayes is especially effective when **the amount of training data is limited,** as it relies on probability estimates rather than complex model training.
- Despite its simplicity, it often performs surprisingly well in real-world classification tasks.

The Bayesian Theorem

- **Bayes' Theorem** explains how to update the probability of a possible cause when new evidence is observed, by combining what we already know (prior knowledge) with how likely the evidence is.

  Bayes' idea: *We update what we believe when we see new evidence.*



Why Naive Bayes is “Naive”

- Naive Bayes is called “naive” because it makes strong assumptions about the data.
  - **It assumes feature independence:**
    The algorithm assumes that all input features are independent of each other given the class, even though in real-world data, this might not hold true as, many features are correlated.
  - **It simplifies real-world relationships:**
    Naive Bayes ignores interactions between features and treats each feature as equally contributing to the final decision, which reduces complexity but may be unrealistic in real world settings.

Naive Bayes Classifiers

- Naive Bayes classifiers differ mainly in how they model feature data, depending on whether the features are continuous, counts, or binary.
  - **Gaussian Naive Bayes:**
    Used when features are **continuous** and assumed to follow a **normal distribution** (e.g., height, temperature).
  - **Multinomial Naive Bayes:**
    Used for **count-based data**, especially in **text classification** (e.g., word frequencies in documents).
  - **Bernoulli Naive Bayes:**
    Used for **binary features**, indicating presence or absence (e.g., a word appears or not in a document).

Support Vector Machine (SVM)

- Support Vector Machine (SVM) is a supervised learning algorithm used mainly for **classification**.
- It works by finding the **best boundary** that separates different classes in the feature space.
- The best boundary is the one that **maximizes the margin**, which is the distance between the boundary and the nearest data points from each class.
- SVM is powerful for **high-dimensional data** and is widely used in tasks like image classification and text categorization.
- SVMs are **memory-efficient** as they focus only on critical data points (support vectors) near the boundary and excel with smaller datasets.

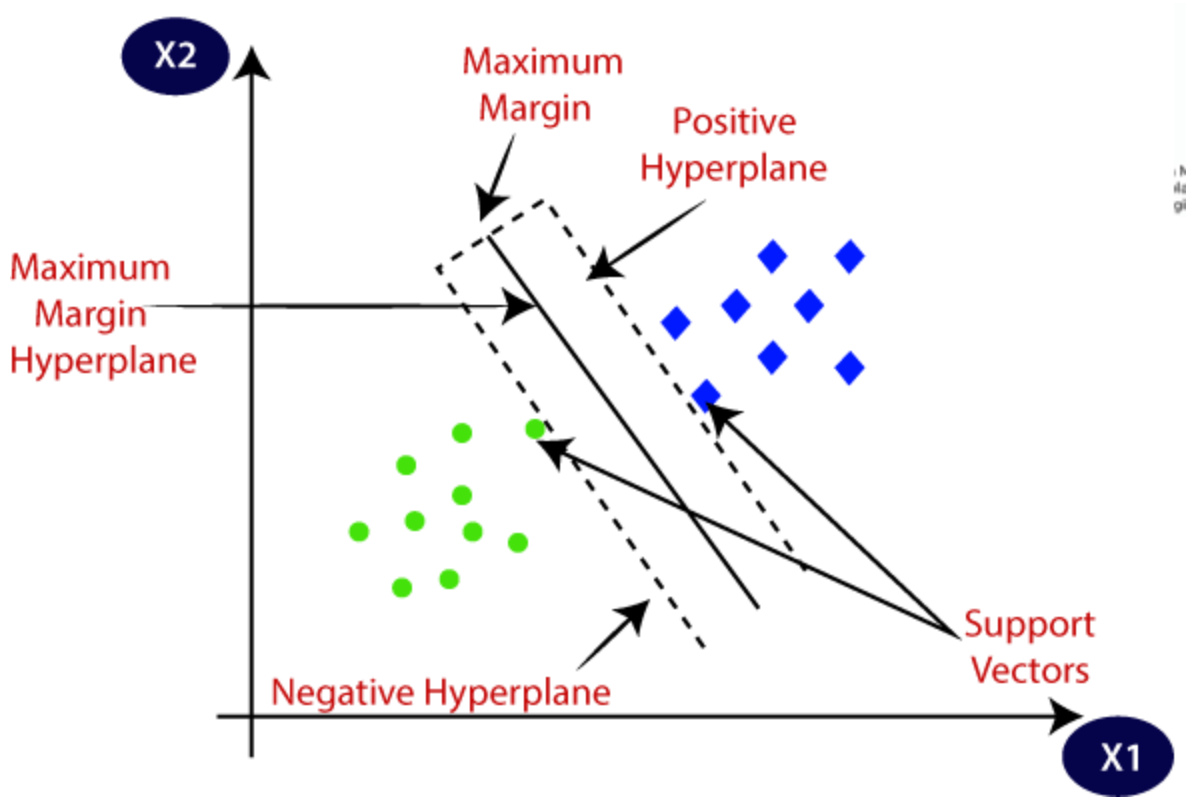
SVM Classifiers

- SVM classifiers can be divided into **linear and non-linear SVM**, depending on whether the data can be separated by a straight boundary.
  - A **linear SVM** is used when the classes can be separated by a straight line (in 2D) or a flat hyperplane (in higher dimensions). It works best when the data is clearly separable and the relationship between features is simple.
  - A **non-linear SVM** is used when the classes are not linearly separable. It uses the **kernel trick** to transform the data into a higher-dimensional space where a linear hyperplane can separate the classes.

How SVM Works

- SVM separates classes using a **hyperplane**, which is a boundary that divides the feature space into two parts.
- The algorithm chooses the hyperplane that maximizes the **margin**, which is the distance between the hyperplane and the closest data points from each class.
- The closest points that define the margin are called **support vectors**, and they are the most important data points because they determine the final decision boundary.
- For data that is not linearly separable, SVM uses the **kernel trick** to transform the data into a higher-dimensional space where a hyperplane can separate the classes.

Support Vector Machine (SVM)



SVM Kernels

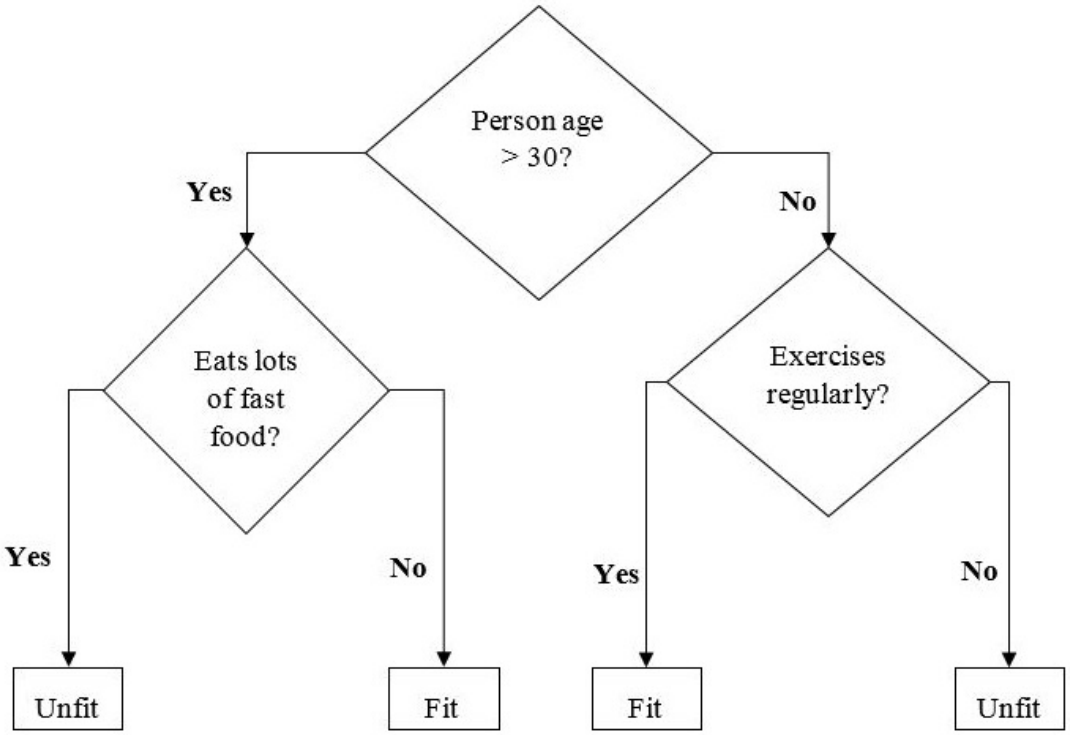
- Kernels are functions used by SVM to transform data into a higher-dimensional space so that non-linear problems become linearly separable. Three Commonly Used Kernels include;
  - **Linear Kernel:**
    Used when data is linearly separable; fast and simple.
  - **Polynomial Kernel:**
    Creates curved decision boundaries by considering polynomial feature combinations.
  - **RBF (Radial Basis Function) Kernel:**
    Most popular for non-linear data; maps data into a very high-dimensional space for complex separation.

Decision Trees

- A **Decision Tree** is a non-parametric supervised learning algorithm that uses a tree-like structure to make decisions based on feature values.
- It splits the data into branches using **if–then rules**, starting from a **root node**, splitting at **internal nodes**, and ending at **leaf nodes** that represent the final class.
- Decision trees are easy to interpret and visualize, making them useful for explaining model decisions, but they can overfit the training data if the tree grows too deep.
- Decision trees are widely used in real-world problems such as loan approval, medical diagnosis, and customer segmentation.

Decision Trees

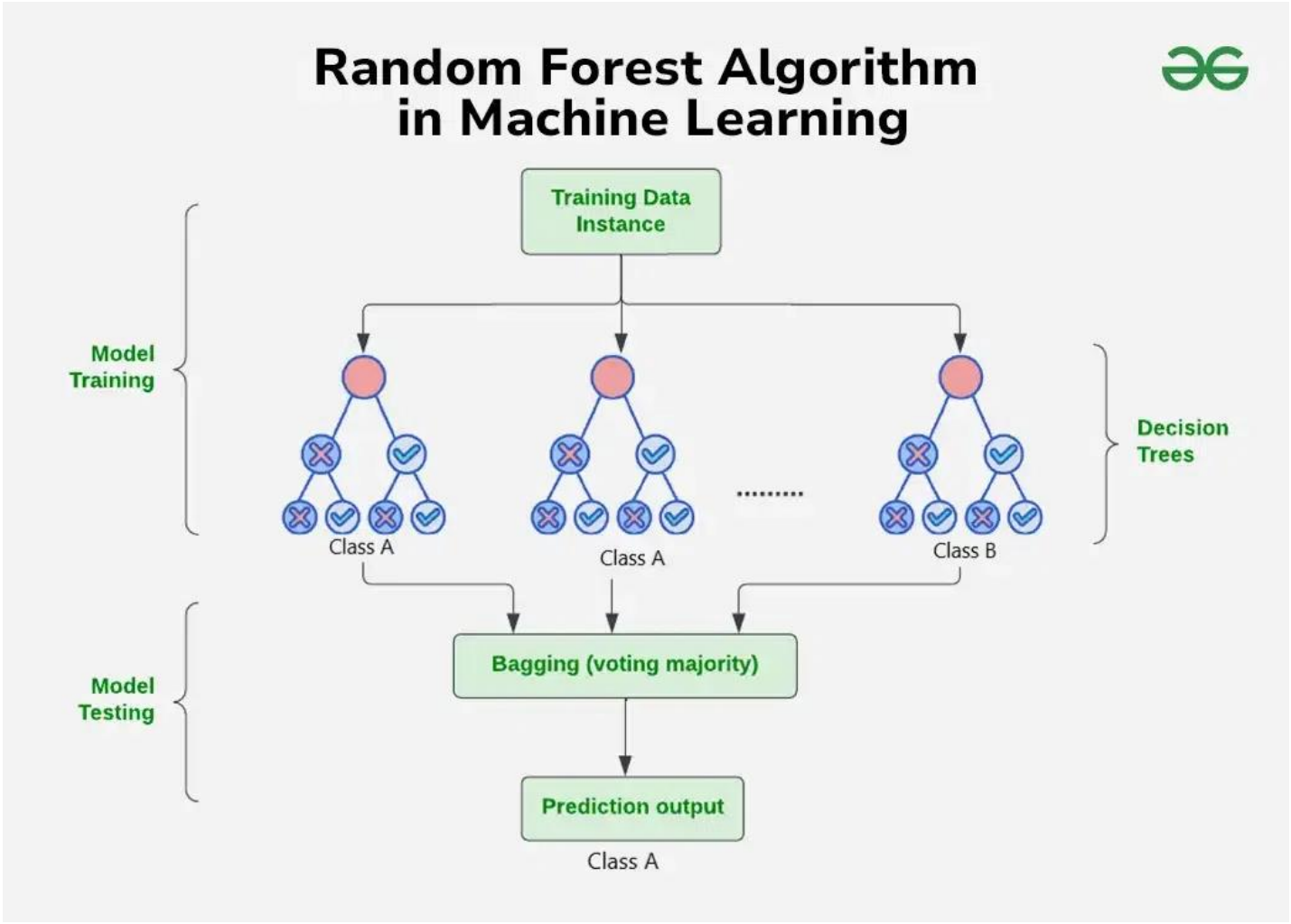
- Is a person fit?



Random Forest

- Random Forest is an **ensemble learning method** that combines many decision trees to improve prediction accuracy.
- Instead of using one tree, Random Forest builds multiple decision trees using different random subsets of the data and features.
- Each tree makes its own prediction, and the final output is determined by **majority voting** (for classification) or averaging (for regression).
- This approach reduces overfitting compared to a single decision tree and makes the model more stable and accurate.
- Random Forest is widely used in real-world tasks like fraud detection, medical diagnosis, and customer prediction.

Random Forest



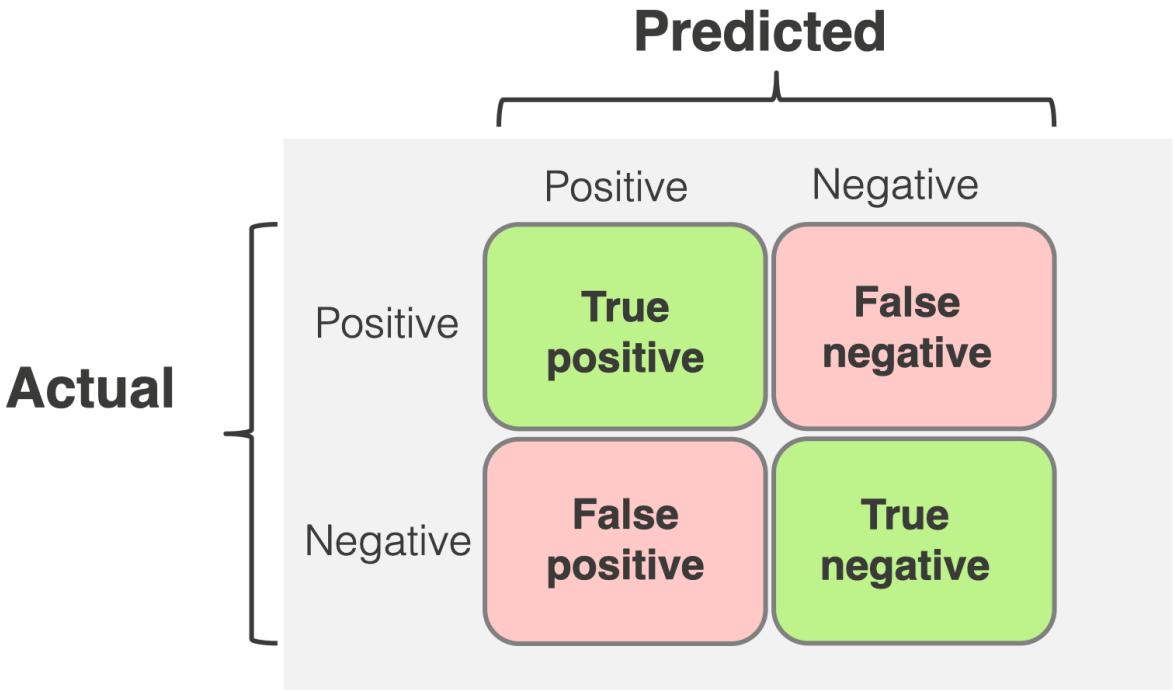
Classification Evaluation Metrics

- In classification, **accuracy alone** can be misleading, especially when classes are imbalanced (one class has far more examples than the other).
- Evaluation metrics help us measure how well a model predicts each class and how often it makes mistakes.
- The most common metrics are based on the **confusion matrix**, which shows the counts of correct and incorrect predictions.
- Using multiple metrics helps us choose the best model for the specific problem and avoid biased results.

Confusion Matrix

- A **confusion matrix** is a table used to evaluate the performance of a classification model by comparing predicted labels with actual labels. It consists of four outcomes:
  - **True Positive (TP):** Model correctly predicts the positive class.
  - **True Negative (TN):** Model correctly predicts the negative class.
  - **False Positive (FP):** Model incorrectly predicts positive when it is actually negative.
  - **False Negative (FN):** Model incorrectly predicts negative when it is actually positive.

Confusion Matrix



Key Classification Metrics

- These four metrics are derived from the **confusion matrix**, which shows how many predictions are correct or incorrect.
  - **Accuracy:** Measures how often the model is correct overall (both positive and negative predictions).
  - **Precision:** Measures how often the model's positive predictions are actually correct.
  - **Recall (Sensitivity):** Measures how well the model identifies actual positive cases.
  - **F1-Score:** Combines precision and recall to give a balanced measure of performance, especially useful when classes are imbalanced.

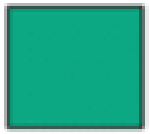
Key Classification Metrics



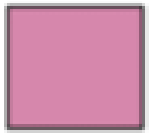
$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$



$$\text{Specificity} = \frac{TN}{TN + FP}$$



$$\text{Precision} = \frac{TP}{TP + FP}$$



$$\text{Recall} = \frac{TP}{TP + FN}$$

Regression Tasks

- Regression in machine learning refers to a supervised learning technique where the goal is to **predict a continuous numerical value** based on one or more independent features.
- It finds relationships between variables so that predictions can be made.
- Instead of predicting categories like in classification, regression models estimate quantities such as price, temperature, or age.
- The goal is to learn the relationship between **input features** and a **numeric target**, then predict values for new data.
- Regression is widely used in real-world tasks like forecasting sales and estimating customer lifetime value.

Regression Assumptions

- Regression models work best when certain conditions about the data are met. These assumptions include;
  - **Linearity:**
    The relationship between the input features and the target variable should be roughly **straight-line** (or linear). If the pattern is curved, the model may not fit well.
  - **Independence of Errors:**
    The prediction errors (differences between actual and predicted values) should be **independent** and not influence each other.
  - **Homoscedasticity (Equal Variance):**
    The spread of errors should be **consistent across all levels** of the predicted values. If errors grow larger at higher values, the model may be unreliable.

Regression Algorithms

- Regression algorithms are used to predict continuous numeric outcomes by learning relationships between input features and the target variable.
- The most common regression methods start with **Linear Regression**, which models a straight-line relationship between inputs and output.
- When the relationship is not linear, **Polynomial Regression** extends linear regression by adding higher-degree feature terms.
- To prevent overfitting and improve model generalization, **Lasso** and **Ridge Regression** apply regularization to penalize large coefficient values.

Linear Regression

- **Linear Regression** is a supervised learning algorithm used to predict a **continuous output** using one or more input features.
- It assumes a **linear relationship** between the input features and the target variable, meaning changes in the inputs result in proportional changes in the output.
- The model learns coefficients (weights) for each feature and fits a straight line (or plane) that minimizes prediction errors.
- Linear regression is commonly used in real-world tasks like predicting house prices, sales forecasting, and estimating growth trends.

Logistic Regression

- **Logistic Regression** is a classification algorithm, but it is built using regression concepts.
- Instead of predicting a continuous value, it predicts the **probability of belonging to a class** (e.g., spam or not spam).
- It uses the **sigmoid function** to map predictions into a range between 0 and 1.
- If the probability is above a threshold (usually 0.5), the model predicts the positive class; otherwise, it predicts the negative class.

Example: Predicting whether a student will pass or fail based on study hours and attendance.

Logistic Regression

- Although it uses a regression-style equation, logistic regression is mainly considered as a classification algorithm for the following two key reasons.
  - **It predicts probabilities, not continuous values:**
    Logistic regression estimates the **probability of an input belonging to a class.** The final result is a probability score rather than a direct numeric prediction.
  - **It converts probability into a class label:**
    The model applies a **sigmoid function** to squash values into a range between 0 and 1, then uses a **threshold** (e.g., 0.5) to assign the final class. This final step makes it a **classification model**.

Polynomial Regression

- **Polynomial Regression** is used when the relationship between features and the target is **non-linear**.
- It extends linear regression by adding **polynomial terms** (squared, cubic, etc.) of the features.
- This allows the model to fit curved patterns in the data while still using a linear model structure.
- However, higher polynomial degrees can lead to **overfitting**, especially with small datasets.

Example: Predicting sales growth where the trend accelerates over time.

Lasso Regression

- **Lasso Regression**, also called as L1 regularization, is a regularized version of linear regression that prevents overfitting.
- It adds a penalty to the model that **shrinks some coefficients to zero**, effectively removing less important features.
- This makes Lasso useful for **feature selection** when dealing with many input variables.
- It is similar to the Ridge Regression, except that penalty term contains only the absolute weights instead of a square of weights.
- Since it takes absolute values, hence, it can shrink the slope to 0, whereas Ridge Regression can only shrink it near to 0.

Ridge Regression

- **Ridge Regression**, is also called as L2 regularization, is another regularized linear regression technique.
- It adds a penalty (a small amount of bias) to the model that **shrinks coefficients**, but unlike Lasso, it does not force them to zero.
- A general linear or polynomial regression will fail if there is high collinearity between the independent variables, so to solve such problems, Ridge regression can be used.
- It helps to solve the problems if we have more parameters than samples.

Evaluating Regression Models

- Regression evaluation metrics measure how close the predicted numeric values are to the actual values.
  - **MAE (Mean Absolute Error):**
    Average absolute difference between predictions and actual values.
  - **MSE (Mean Squared Error):**
    Average squared difference; penalizes large errors more.
  - **RMSE (Root Mean Squared Error):**
    Square root of MSE; error measured in the same units as the target.
  - **R^2 (R-squared):**
    Shows how much variation in the target is explained by the model.

Applications of Supervised Learning

- Supervised learning is widely used in real-world tasks where labeled data is available.
- Common applications include **spam detection**, **disease diagnosis**, **loan approval**, and **customer churn prediction**.
- It is also used in **image recognition**, **speech recognition**, and **language translation**.
- Supervised learning helps businesses make data-driven decisions by predicting outcomes based on historical data.
- It is especially powerful in areas where past outcomes are known and can be used to train accurate prediction models.

Limitations of Supervised Learning

- Supervised learning requires **large amounts of labeled data**, which can be expensive and time-consuming to obtain.
- It may **overfit** when the model becomes too complex or the dataset is small.
- Models may struggle when the data changes over time (**concept drift**), requiring frequent retraining.
- It performs poorly when the labels are noisy or incorrect, reducing prediction accuracy.
- It also cannot learn **without labeled data**, making it unsuitable for tasks where labels are unavailable or difficult to create.